



endingo
정훈이의 공부일지

CATEGORY



👋 안녕하세요, AI 개발자 이정훈입니다

AI · 백엔드 · 데이터 분석 중심의 프로젝트와 경험을 공유합니다.

 Contact Me

Data science/자연어처리

[자연어 처리] 1. 토큰화

endingo 2025. 4. 12. 19:10

1. 토큰화

토큰:

문장의 가장 작은 단위

토큰화:

문장을 토큰 시퀀스로 나누는 과정

2. 형태소 분석(한글)

토큰화보다 더 깊은 분석

단어를 구성하는 형태소를 식별하고 분류

형태소 분석기(한글): KoNLPy, Kiwi



정훈이의 공부일지



```
text_ko2 = "나는 ChatGPT를 사용한다."
```

1) KoNLPy 사용

- 형태소 분석기 선언

```
from konlpy.tag import Okt
```

```
from konlpy.tag import Okt
```

```
# 형태소 분석기 불러오기
```

```
okt = Okt()
```

- 토큰화

```
okt.morphs(text_ko1): 형태소 단위 토큰화
```

```
okt.pos(text_ko1): 품사 태깅 포함
```

```
# 형태소 분석 (품사 태깅 포함)
```

```
morphs = okt.morphs(text_ko1) # 형태소 단위 토큰화
```

```
pos = okt.pos(text_ko1) # 품사 태깅 포함
```

```
print("형태소 단위 토큰화:", morphs)
```

```
print("품사 태깅:", pos)
```

```
형태소 단위 토큰화: ['아버지', '가방', '에', '들어가신다', '.', '']
품사 태깅: [('아버지', 'Noun'), ('가방', 'Noun'), ('에', 'Josa'), ('들어가신다', 'Verb'), ('.', 'Punctuation')]
```

2) kiwi 사용

- 형태소 분석기 선언

```
from kiwipiepy import Kiwi
```

```
from kiwipiepy import Kiwi
```

```
# Kiwi 선언
```

```
kiwi = Kiwi()
```

- 토큰화

```
kiwi.tokenize(text_ko1): 형태소 분석
```

```
token.tag: 품사 태깅
```

```
# 형태소 분석 (품사 태깅 포함)
```

```
tokens = kiwi.tokenize(text_ko1)
```



정훈이의 공부일지



for token in tokens:

```
print(token) # 형태소, 시작 위치, 끝 위치, 품사, 확률 정보 출력
```

품사 태깅만 출력

```
pos_tags = [(token.form, token.tag) for token in tokens]
```

```
print("\n품사 태깅:", pos_tags)
```

- 사전 추가

사전에 없는 단어는 새로 추가해줘야한다.

사전 추가하기전 결과

```
text_ko3 = "나는 국물을 익힌다."
```

형태소 분석

```
tokens = kiwi.tokenize(text_ko3)
```

품사 태깅만 출력

```
pos_tags = [(token.form, token.tag) for token in tokens]
```

```
print("\n사용자 사전 적용 후 품사 태깅:", pos_tags)
```

```
사용자 사전 적용 후 품사 태깅: [('나', 'NP'), ('는', 'JX'), ('국', 'NNG'), ('물', 'NNG'), ('을', 'JKO'), ('익히', 'VV'), ('다', 'EF'), ('.', '.')]

```

```
kiwi.add_user_word("국물", "NNG")
```

사용자 사전 추가

```
kiwi.add_user_word("국물", "NNG") # "국물"을 일반명사(NNG)로 등록
```

형태소 분석

```
tokens = kiwi.tokenize(text_ko3)
```

품사 태깅만 출력

```
pos_tags = [(token.form, token.tag) for token in tokens]
```

```
print("\n사용자 사전 적용 후 품사 태깅:", pos_tags)
```

```
('국물', 'NNG'),

```

3. 형태소 분석(영문): NLTK, spaCy

- NLTK: 교육용, 연구용 속도느리고 부족한점 많아 학습용

- spaCy: 빠르고 효율적, 대규모 NLP 작업

1) 입력문장



정훈이의 공부일지



word_tokenize(text_en1): 토큰화

pos_tag(tokens): 품사 태깅

WordNetLemmatizer(): Lemmatizer 적용 => 기본형 변형

토큰화

tokens = word_tokenize(text_en1)

품사 태깅

nltk_pos = pos_tag(tokens)

Lemmatizer 적용(잘 안됨.)

lemmatizer = WordNetLemmatizer()

lemmas = [(word, lemmatizer.lemmatize(word)) for word, pos in nltk_pos]

결과 출력

print("토큰화 :", tokens)

print("품사 태깅 :", nltk_pos)

print("기본형 변환(lemma):", lemmas)

```

토큰화      : ['I', 'am', 'running', 'towards', 'the', 'beautiful', 'park', '.']
품사 태깅   : [('I', 'PRP'), ('am', 'VBP'), ('running', 'VBG'), ('towards', 'IN'), ('the', 'the')]
기본형 변환(lemma): [('I', 'I'), ('am', 'am'), ('running', 'running'), ('towards', 'towards'),

```

3) spaCy 토큰화

spacy(text_en1)

token.text: 토큰화된거보여주기

token.pos_: 품사태깅보여주기

token.lemma_: 기본형으로 변환해 보여주기

형태소 분석 실행

spacy_docs = spacy(text_en1)

tokens = [token.text for token in spacy_docs]

pos_tags = [(token.text, token.pos_) for token in spacy_docs]

lemmas = [(token.text, token.lemma_) for token in spacy_docs]

결과 출력

print("토큰화 :", tokens)

print("품사 태깅 :", pos_tags)

print("기본형 변환(lemma):", lemmas)



정훈이의 공부일지



am => be, running => run

4. 서브워드 단위 토큰화: BPE, Wordpiece

- 형태소 분석보다는 토큰화 초점
- BPE: GPT 모델에서 사용
- Wordpiece: BERT 모델에서 사용 BPE보다 정교한 사전 구축

1) BPE

- 토크나이저 다운로드

```
AutoTokenizer.from_pretrained("gpt2")
```

```
# GPT 기반 BPE 토크나이저 로드
```

```
bpe = AutoTokenizer.from_pretrained("gpt2")
```

- 토큰화

```
bpe.tokenize(subword_text1)
```

```
tokens_bpe = bpe.tokenize(subword_text1)
```

```
print("BPE Tokenized:", tokens_bpe)
```

```
BPE Tokenized: ['I', 'Ġam', 'Ġlearning', 'ĠByte', 'ĠPair', 'ĠEnc', 'Ġoding', 'Ġand', 'ĠWord', 'p
```

2) Wordpiece

- 토크나이저 다운로드

```
AutoTokenizer.from_pretrained("bert-base-uncased")
```

```
# BERT 기반 WordPiece 토크나이저 로드
```

```
wordpiece = AutoTokenizer.from_pretrained("bert-base-uncased")
```

- 토큰화

```
wordpiece.tokenize(subword_text1)
```

```
tokens_wp = wordpiece.tokenize(subword_text1)
```

```
print("wp Tokenized:", tokens_wp)
```

```
wp Tokenized: ['I', 'am', 'learning', 'byte', 'pair', 'encoding', 'and', 'word', '##piece', '.']
```



정훈이의 공부일지



1) BPE

`bpe.convert_tokens_to_idx(tokens_bpe)`

각 토큰을 숫자로 변환 (토큰 ID 부여)

`ids_bpe = bpe.convert_tokens_to_ids(tokens_bpe)``print("BPE Tokenized:", tokens_bpe)``print("Token IDs :", ids_bpe)`

```
BPE Tokenized: ['Token', 'ization', 'Gallows', 'Gus', 'Gto', 'Gefficiently', 'Gprocess', 'Gcomp']
Token IDs      : [30642, 1634, 3578, 514, 284, 18306, 1429, 3716, 2456, 588, 11241, 1634, 290, 61]
```

2) wordpiece

`wordpiece.convert_tokens_to_ids(tokens_wp)`

각 토큰을 숫자로 변환 (토큰 ID 부여)

`ids_wp = wordpiece.convert_tokens_to_ids(tokens_wp)``print("WP Tokenized:", tokens_wp)``print("Token IDs :", ids_wp)`

```
WP Tokenized: ['token', '##ization', 'allows', 'us', 'to', 'efficiently', 'process', 'complex']
Token IDs      : [19204, 3989, 4473, 2149, 2000, 18228, 2832, 3375, 2616, 2066, 19204, 3989, 199]
```

공감

구독하기

[Data science](#) > [자연어처리](#) 카테고리의 다른 글[\[자연어처리\] 3. RNN 개념 이해](#) (0)[\[자연어처리\] 2. 임베딩](#) (0)

정훈이의 공부일지

개발하면서 겪는 시행착오와 배움을 기록합니다. 백엔드, AI, 데이터 분석, 프로젝트 경험 중심으로 운영 중입니다. 공부하면서 얻은 인사이트를 함께 나누고 싶어요!

구독하기



정훈이의 공부일지



이름

비밀번호

로그인 댓글만 허용한 블로그입니다

등록

공지사항

최근에 올라온 글

- [자연어처리] 3. RNN 개념 이해
- [자연어처리] 2. 임베딩
- [자연어 처리] 1. 토큰화
- [이미지처리] 5. YOLO 모델

최근에 달린 댓글

덕분에 좋은 정보 얻어 갑니다 ! ...

Total

218

Today	1
Yesterday	1

링크

TAG

- github
- git

more

« 2025/04 »						
일	월	화	수	목	금	토
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30			

글 보관함

- 2025/04 (27)
- 2025/03 (7)

Blog is powered by Tistory / Designed by Tistory

이정훈 개발자

AI와 백엔드, 데이터 분석을 좋아하는 개발자입니다.
다양한 문제 해결과 창의적인 개발을 추구합니다.
 이메일 보내기